CONFIGURATION & RUNTIME

Title: Project Architecture & Environment Setup

System manifest, dependency pinning, and environment key management for ComicCraft.

> **Runtime Specification:** ComicCraft runs on Python 3.10+ using FastAPI for asynchronous HTTP serving, Hugging Face Diffusers for local image inference via PyTorch, and Google Generative AI for multimodal prompt generation.

requirements.txt — PIP DEPENDENCIES

```
fastapi>=0.110.0
uvicorn[standard]>=0.28.0
jinja2>=3.1.3
python-multipart>=0.0.9
google-generativeai>=0.5.0
diffusers>=0.27.0
transformers>=4.38.0
torch>=2.2.0
accelerate>=0.27.0
Pillow>=10.2.0
fpdf2>=2.7.8
python-dotenv>=1.0.1
pydantic>=2.6.0
```

.env — CONFIGURATION CREDENTIALS

```
# Google Gemini AI Authentication Key
GEMINI_API_KEY=your_gemini_api_key_here

# Hugging Face Access Token for Diffusers Pipeline
HF_API_KEY=your_huggingface_api_key_here
```

VS Code Terminal Setup Commands — SHELL SCRIPT

```
# Step 1: Create and activate isolated environment
python -m venv comiccraft-env
source comiccraft-env/bin/activate       # Linux/macOS
comiccraft-env\Scripts\activate          # Windows PowerShell

# Step 2: Install packages & start dev server
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

BACKEND CORE

Title: Application Entry Point (`app/main.py`)

Initialization of the FastAPI application lifecycle, static mounts, and route aggregation.

> **Component Role:** Instantiates the root ASGI server instance, automatically registers storage mounts for panel renderings and generated PDFs, and attaches the master application router.

app/main.py PYTHON MODULE

```python
import os
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from app.routes import router

# Initialize FastAPI root instance
app = FastAPI(
    title="ComicCraft - AI Comic Story Creator",
    description="Automated Comic Strip & Graphic Novel Generator via Gemini & Stable Diffusion",
    version="1.0.0"
)

# Pre-create output directories for images and compiled PDF documents
os.makedirs("static/panels", exist_ok=True)
os.makedirs("static/exports", exist_ok=True)
os.makedirs("static/fonts", exist_ok=True)

# Mount the static directory to serve images and export assets
app.mount("/static", StaticFiles(directory="static"), name="static")

# Register all route definitions
app.include_router(router)

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("app.main:app", host="127.0.0.1", port=8000, reload=True)
```

Directory / Mount Point	Target Directory	Description
`/static/panels`	`static/panels/`	Stores raster images generated for comic panels.
`/static/exports`	`static/exports/`	Stores compiled multi-page PDF comic releases.

BACKEND ROUTING

Title: Application Routes & Controller (`app/routes.py`)

HTTP form intake, JSON API endpoints, AI pipeline invocation, and template dispatch.

app/routes.py PYTHON MODULE

```python
import os, traceback
from fastapi import APIRouter, Request, Form, HTTPException
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel

from app.gemini_flash import generate_outline
from app.gemini_pro import generate_story
from app.image_generator import generate_image
from app.layout_builder import build_comic_layout
from app.exporters import save_pdf

router = APIRouter()
templates = Jinja2Templates(directory="templates")

class PromptRequest(BaseModel):
    prompt: str
    character_name: str
    setting: str
    tone: str
    style: str

@router.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})

@router.post("/generate", response_class=HTMLResponse)
async def generate_comic(
    request: Request,
    prompt: str = Form(...),
    character_name: str = Form(...),
    setting: str = Form(...),
    tone: str = Form(...),
    style: str = Form(...)
):
    try:
        full_prompt = (
            f"{prompt}\\n"
            f"The main character is {character_name}. "
            f"The setting is a {setting}. "
            f"The tone is {tone}. The art style is {style}."
        )
        outline = generate_outline(full_prompt)
        if not isinstance(outline, list) or not all("image_prompt" in p for p in outline):
            raise ValueError("Invalid outline structure from Gemini Flash.")

        full_story = generate_story(outline)
        images = [generate_image(f"{p['image_prompt']}, {style} style") for p in outline]
        layout = build_comic_layout(images, full_story, outline)

        pdf_path = save_pdf(layout)
        web_pdf_path = "/" + pdf_path.replace("\\\\", "/")

        return templates.TemplateResponse("comic_preview.html", {
            "request": request,
            "layout": layout,
            "pdf_path": web_pdf_path
        })
    except Exception as e:
        traceback.print_exc()
        raise HTTPException(status_code=500, detail=str(e))
```

AI PIPELINES

Title: Story & Panel Generation (`gemini_flash.py` & `gemini_pro.py`)

Structured panel planning with Gemini 1.5 Flash and dialogue synthesis with Gemini 1.5 Pro.

app/gemini_flash.py — GEMINI FLASH PLANNER

```python
import json, os
import google.generativeai as genai
from dotenv import load_dotenv

load_dotenv()
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
model = genai.GenerativeModel("models/gemini-1.5-flash")

def generate_outline(user_prompt: str) -> list:
    prompt = f"""You are a professional AI comic planner.
Generate a strictly formatted JSON array containing 5 panel descriptions for:
STORY: "{user_prompt}"

Each JSON object must include:
- "panel" (integer), "title" (string), "scene_description" (string), "image_prompt" (string)
Respond ONLY in valid JSON format without explanations or markdown backticks."""

    try:
        response = model.generate_content(prompt)
        text = response.text.strip()
        if text.startswith("```json"):
            text = text.replace("```json", "").replace("```", "").strip()
        elif text.startswith("```"):
            text = text.replace("```", "").strip()
        return json.loads(text)
    except Exception as e:
        return [{"error": f"Failed to generate outline: {str(e)}"}]
```

app/gemini_pro.py — GEMINI PRO STORY WRITER

```python
import os
import google.generativeai as genai
from dotenv import load_dotenv

load_dotenv()
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
model = genai.GenerativeModel("models/gemini-1.5-pro")

def generate_story(outline: list) -> str:
    formatted = "\\n".join([f"{i+1}. {item}" for i, item in enumerate(outline)])
    prompt = f"""You're a comic book writer. Write an engaging comic story with dialogues.
Panel Outline:\\n{formatted}\\n
Guidelines:
- Fun, engaging tone.
- Mark panels strictly with '**Panel X: Title**' followed by narration and dialogue."""

    try:
        response = model.generate_content(prompt)
        return response.text
    except Exception as e:
        return f"Error generating story: {str(e)}"
```

GENERATION & INTEGRATION

Title: Visual Generation & Layout Assembly

Implementation of `image_generator.py` and `layout_builder.py` modules.

app/image_generator.py — STABLE DIFFUSION CORE

```
import os, re, torch
from diffusers import StableDiffusionPipeline
from PIL import Image, ImageDraw

device = "cuda" if torch.cuda.is_available() else "cpu"
pipe = None

def get_pipeline():
    global pipe
    if pipe is None:
        dtype = torch.float16 if torch.cuda.is_available() else torch.float32
        pipe = StableDiffusionPipeline.from_pretrained("runwayml/stable-diffusion-v1-5", torch_dtype=dtyp
e).to(device)
    return pipe

def generate_image(prompt: str, filename: str = None) -> str:
    if not filename:
        sanitized = re.sub(r'[^a-zA-Z0-9_-]', '_', prompt[:26].strip())
        filename = f"{sanitized}_{os.urandom(4).hex()}.png"

    out_dir = os.path.join("static", "panels")
    os.makedirs(out_dir, exist_ok=True)
    target = os.path.join(out_dir, filename)

    try:
        pipeline = get_pipeline()
        img = pipeline(prompt, num_inference_steps=25).images[0]
        img.save(target)
    except Exception:
        fallback = Image.new("RGB", (512, 512), color=(15, 23, 42))
        d = ImageDraw.Draw(fallback)
        d.text((20, 240), f"Prompt Rendered:\\n{prompt[:35]}...", fill=(241, 245, 249))
        fallback.save(target)
    return target
```

app/layout_builder.py — DATA TRANSFORMER

```
def build_comic_layout(image_paths: list, full_story: str, outline: list) -> list:
    story_panels = full_story.split("**Panel")
    story_panels = [f"**Panel{p}" for p in story_panels if p.strip()]

    layout = []
    for idx, (img_path, raw_text, meta) in enumerate(zip(image_paths, story_panels, outline), start=1):
        lines = raw_text.strip().splitlines()
        content_lines = lines[1:] if len(lines) > 1 else lines
        layout.append({
            "panel": idx,
            "title": meta.get("title", f"Panel {idx}"),
            "image_path": img_path,
            "text": "\\n".join(content_lines).strip(),
            "scene_description": meta.get("scene_description", "")
        })
    return layout
```

DOCUMENT COMPILER

Title: PDF Publication Engine (`app/exporters.py`)

Vector compilation of layout elements, imagery, and text using FPDF2.

> **Publishing Logic:** Generates uniform, printable pages where each comic panel receives a dedicated visual page with centered title headers, embedded illustration frames, scene contexts, and sanitized narrative text.

app/exporters.py — FPDF2 RENDER ENGINE

```python
import os
from datetime import datetime
from fpdf import FPDF

EXPORT_DIR = os.path.join("static", "exports")
os.makedirs(EXPORT_DIR, exist_ok=True)

class ComicPDF(FPDF):
    pass

def save_pdf(layout: list) -> str:
    pdf = ComicPDF()
    pdf.set_auto_page_break(auto=True, margin=15)

    for item in layout:
        pdf.add_page()

        # Panel Title Header
        pdf.set_font("Helvetica", style="B", size=15)
        pdf.cell(0, 10, f"Panel {item['panel']}: {item['title']}", ln=True, align="C")

        # Panel Artwork Placement
        img_y = 28
        img_h = 100
        if item["image_path"] and os.path.exists(item["image_path"]):
            pdf.image(item["image_path"], x=15, y=img_y, w=pdf.epw, h=img_h)
        else:
            pdf.set_y(img_y)
            pdf.multi_cell(0, 10, f"Artwork missing: {item['image_path']}")

        # Scene Description Metadata
        pdf.set_y(img_y + img_h + 10)
        pdf.set_font("Helvetica", style="I", size=9)
        pdf.multi_cell(0, 6, f"Scene: {item['scene_description']}")
        pdf.ln(3)

        # Narration & Dialogue Flow
        pdf.set_font("Helvetica", size=10)
        clean_narrative = item["text"].encode("latin-1", "replace").decode("latin-1")
        pdf.multi_cell(0, 6, clean_narrative)

    target_file = f"comic_{datetime.now().strftime('%Y%m%d_%H%M%S')}.pdf"
    full_output_path = os.path.join(EXPORT_DIR, target_file)
    pdf.output(full_output_path)
    return full_output_path
```

FRONTEND INTERFACE

Title: Comic Studio Input Form (`templates/index.html`)

User entry interface for comic prompts, character tags, setting, tone, and style selection.

templates/index.html — JINJA2 / HTML5 TEMPLATE

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>ComicCraft - Create Your Comic</title>
  <style>
    body {
      margin: 0; font-family: 'Segoe UI', system-ui, sans-serif;
      background: #0f172a; color: #f8fafc;
      display: flex; justify-content: center; align-items: center; min-height: 100vh;
    }
    .form-container {
      background: #1e293b; padding: 28px 36px; border-radius: 10px;
      box-shadow: 0 10px 30px rgba(0,0,0,0.5); width: 100%; max-width: 480px;
    }
    h2 { margin-top: 0; text-align: center; color: #38bdf8; }
    label { font-size: 13px; font-weight: 600; color: #cbd5e1; display: block; margin-top: 12px; }
    input, textarea, select {
      width: 100%; padding: 9px; margin-top: 5px; border-radius: 5px;
      border: 1px solid #475569; background: #0f172a; color: #fff; box-sizing: border-box;
    }
    button {
      margin-top: 20px; width: 100%; padding: 12px; background: #0284c7;
      color: white; border: none; border-radius: 6px; font-size: 15px;
      font-weight: 700; cursor: pointer;
    }
    button:hover { background: #0369a1; }
  </style>
</head>
<body>
  <div class="form-container">
    <h2>🎨 Create Your Comic</h2>
    <form action="/generate" method="post">
      <label>Story Prompt:</label>
      <textarea name="prompt" rows="3" required placeholder="A brave fox explores..."></textarea>

      <label>Character Name:</label>
      <input type="text" name="character_name" required placeholder="Free" />

      <label>Setting:</label>
      <select name="setting">
        <option value="Forest">Forest</option>
        <option value="School">School</option>
        <option value="Space">Space</option>
        <option value="City">City</option>
      </select>

      <label>Story Tone:</label>
      <select name="tone">
        <option value="Dramatic">Dramatic</option>
        <option value="Funny">Funny</option>
        <option value="Light-hearted">Light-hearted</option>
      </select>

      <label>Art Style:</label>
      <select name="style">
        <option value="Realistic">Realistic</option>
        <option value="Anime">Anime</option>
        <option value="Comic Book">Comic Book</option>
      </select>

      <button type="submit">Generate Comic Strip</button>
    </form>
```

FRONTEND VIEWS

Title: Comic Preview & Export Verification

Templates for interactive preview (`comic_preview.html`) and download confirmation (`export_success.html`).

templates/comic_preview.html — PREVIEW VIEWER

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Preview Your Comic</title>
  <style>
    body { background: #f8fafc; font-family: system-ui, sans-serif; padding: 24px; text-align: center; }
    .panel { background: #fff; max-width: 620px; margin: 0 auto 24px; padding: 20px; border-radius: 8px; b
order: 1px solid #cbd5e1; }
    .panel img { width: 100%; border-radius: 6px; }
    .btn { display: inline-block; padding: 12px 24px; background: #0284c7; color: #fff; text-decoration: n
one; border-radius: 6px; font-weight: bold; }
  </style>
</head>
<body>
  <h1>Your Comic Preview</h1>
  {% for p in layout %}
  <div class="panel">
    <h3>Panel {{ p.panel }}: {{ p.title }}</h3>
    <img src="/{{ p.image_path }}" alt="Scene {{ p.panel }}" />
    <p style="font-style:italic; color:#64748b;">{{ p.scene_description }}</p>
    <p style="white-space:pre-wrap; text-align:left;">{{ p.text }}</p>
  </div>
  {% endfor %}
  <a class="btn" href="/export-success?pdf_path={{ pdf_path }}">Download Your Comic as PDF</a>
</body>
</html>
```

templates/export_success.html — CONFIRMATION PAGE

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Comic Exported Successfully</title>
  <style>
    body { height: 100vh; margin: 0; display: flex; align-items: center; justify-content: center; font-fam
ily: system-ui, sans-serif; background: #f1f5f9; }
    .box { text-align: center; background: white; padding: 40px; border-radius: 10px; border: 1px solid #c
bd5e1; }
    .btn { display: inline-block; padding: 10px 20px; border-radius: 6px; text-decoration: none; font-weig
ht: bold; margin: 6px; }
    .primary { background: #0284c7; color: white; } .secondary { background: #e2e8f0; color: #0f172a; }
  </style>
</head>
<body>
  <div class="box">
    <h2 style="color:#16a34a; margin-top:0;">✅ Comic Exported Successfully!</h2>
    <p>Your AI-powered comic has been successfully generated and compiled into PDF.</p>
    {% if pdf_path %}<a class="btn primary" href="{{ pdf_path }}" download>📥 Open Downloaded PDF</a>{% en
dif %}
    <a class="btn secondary" href="/">🎨 Go Create Another Comic</a>
  </div>
</body>
</html>
```